DL_LAB6

資工碩一

ID : 314551147

Name : 楊承恩

1. Introduction

In this lab, I used DDPM (Denoising Diffusion Probabilistic Model) to perform conditional image generation on the dataset. To jointly consider time embedding and condition embedding, I designed a UNet architecture enhanced with CBAM, which was previously applied in Lab 2, to strengthen the model's focus on critical features.

The training objective is based on MSE loss for regressing the true noise, and I compared both linear and cosine noise schedules. To further improve generation quality, I incorporated conditional dropout, and applied Classifier-Free Guidance (CFG) during sampling, enabling the model to flexibly control the influence of conditions on the generated results.

2. Implementation details

ICLEVRDataset:

It is used to read train.json and object.json, load each image with its corresponding object labels, and convert them into one-hot vectors. The images are then resized to 64×64 and normalized.

```python
class ICLEVRDataset(Dataset):
    def __init__(self, data_root, train_json, objects_json, image_size=64):
        super().__init__()
        self.data_root = data_root
        self.mapping = load_objects(objects_json)
        self.num_classes = 24
        with open(train_json, 'r', encoding='utf-8') as f:
            self.meta = json.load(f)
        self.filenames = list(self.meta.keys())
        self.transform = transforms.Compose([
            transforms.Resize((image_size, image_size), interpolation=Image.BILINEAR),
            transforms.ToTensor(),
            transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)),
        ])

    def __len__(self):
        return len(self.filenames)

    def __getitem__(self, idx):
        fname = self.filenames[idx]
        labels = self.meta[fname]
        img_path = os.path.join(self.data_root, fname)
        img = Image.open(img_path).convert('RGB')
        x = self.transform(img)
        y = multi_labels_to_onehot(labels, self.mapping, num_classes=24)
        return x, y
```

timestep_embedding:

The diffusion timestep t is converted into a sinusoidal (sin/cos) positional embedding, forming a fixed-dimensional vector that provides the model with information about the current denoising step.

```python
def timestep_embedding(timesteps, dim, max_period=10000):
    half = dim // 2
    freqs = torch.exp(-math.log(max_period) * torch.arange(0, half, dtype=torch.float32, device=timesteps.device) / half)
    args = timesteps.float().unsqueeze(1) * freqs.unsqueeze(0)
    emb = torch.cat([torch.cos(args), torch.sin(args)], dim=-1)
    if dim % 2:
        emb = torch.cat([emb, torch.zeros_like(emb[:, :1])], dim=-1)
    return emb
```

ChannelAttention, SpatialAttention, CBAM:

Channel attention and spatial attention are applied sequentially to enhance the model's focus on important features and regions, thereby improving the quality of the generated images.

```python
class ChannelAttention(nn.Module):
    def __init__(self, in_channels, reduction_ratio=16):
        super().__init__()
        self.global_avg_pool = nn.AdaptiveAvgPool2d(1)
        self.global_max_pool = nn.AdaptiveMaxPool2d(1)
        hidden = max(in_channels // reduction_ratio, 8)
        self.fc = nn.Sequential(
            nn.Linear(in_channels, hidden, bias=False),
            nn.ReLU(inplace=True),
            nn.Linear(hidden, in_channels, bias=False)
        )
        self.sigmoid = nn.Sigmoid()
    def forward(self, x):
        b, c, _, _ = x.size()
        avg_pool = self.fc(self.global_avg_pool(x).view(b, c)).view(b, c, 1, 1)
        max_pool = self.fc(self.global_max_pool(x).view(b, c)).view(b, c, 1, 1)
        attention = self.sigmoid(avg_pool + max_pool)
        return x * attention

class SpatialAttention(nn.Module):
    def __init__(self, kernel_size=7):
        super().__init__()
        padding = (kernel_size - 1) // 2
        self.conv = nn.Conv2d(2, 1, kernel_size=kernel_size, padding=padding, bias=False)
        self.sigmoid = nn.Sigmoid()
    def forward(self, x):
        avg_out = torch.mean(x, dim=1, keepdim=True)
        max_out = torch.max(x, dim=1, keepdim=True)[0]
        pool_out = torch.cat([avg_out, max_out], dim=1)
        attention = self.sigmoid(self.conv(pool_out))
        return x * attention

class CBAM(nn.Module):
    def __init__(self, in_channels, reduction_ratio=16, kernel_size=7):
        super().__init__()
        self.channel_attention = ChannelAttention(in_channels, reduction_ratio)
        self.spatial_attention = SpatialAttention(kernel_size)
    def forward(self, x):
        x = self.channel_attention(x)
        x = self.spatial_attention(x)
        return x
```

```python
class ResidualBlock(nn.Module):
    def __init__(self, in_ch, out_ch, emb_ch, use_cbam: bool = True):
        super().__init__()
        self.norm1 = _make_gn(in_ch)
        self.act1  = nn.SiLU()
        self.conv1 = nn.Conv2d(in_ch, out_ch, 3, padding=1)

        self.norm2 = _make_gn(out_ch)
        self.act2  = nn.SiLU()
        self.conv2 = nn.Conv2d(out_ch, out_ch, 3, padding=1)

        self.emb_proj = nn.Linear(emb_ch, out_ch * 2)  # FiLM: scale, shift
        self.skip = nn.Conv2d(in_ch, out_ch, 1) if in_ch != out_ch else nn.Identity()
        self.cbam = CBAM(out_ch) if use_cbam else nn.Identity()

    def forward(self, x, emb):
        h = self.conv1(self.act1(self.norm1(x)))
        scale, shift = self.emb_proj(emb).chunk(2, dim=1)
        h = self.norm2(h)
        h = h * (1 + scale[:, :, None, None]) + shift[:, :, None, None]
        h = self.conv2(self.act2(h))
        h = h + self.skip(x)
        h = self.cbam(h)
        return h
```

Residual Block:

The model is built on a two-layer convolutional structure with residual (skip) connections, employing Group Normalization and the SiLU activation function.

A FiLM mechanism is introduced: time and condition embeddings are linearly projected and decomposed into scale and shift parameters, which directly modulate the feature maps.

Additionally, a CBAM module is integrated to enhance the model's channel and spatial attention.

This design allows the model to jointly fuse temporal information and conditional control, thereby improving its ability to capture and emphasize critical features.

```python
class UNetCond(nn.Module):
    def __init__(self, in_ch: int = 3, out_ch: int = 3, emb_dim: int = 256, num_classes: int = 24):
        super().__init__()
        self.time_mlp = nn.Sequential(
            nn.Linear(emb_dim, emb_dim), nn.SiLU(), nn.Linear(emb_dim, emb_dim)
        )
        self.cond_mlp = nn.Sequential(
            nn.Linear(num_classes, emb_dim), nn.SiLU(), nn.Linear(emb_dim, emb_dim)
        )
        self.inc   = ResidualBlock(in_ch,   64,  emb_dim)
        self.down1 = Down(64,  128, emb_dim)
        self.down2 = Down(128, 256, emb_dim)
        self.down3 = Down(256, 512, emb_dim)
        self.down4 = Down(512, 512, emb_dim)
        self.up1 = Up(in_ch=512, out_ch=256, emb_ch=emb_dim, skip_ch=512)
        self.up2 = Up(in_ch=256, out_ch=128, emb_ch=emb_dim, skip_ch=256)
        self.up3 = Up(in_ch=128, out_ch= 64, emb_ch=emb_dim, skip_ch=128)
        self.up4 = Up(in_ch= 64, out_ch= 64, emb_ch=emb_dim, skip_ch= 64)
        self.out_norm = _make_gn(64)
        self.out_act  = nn.SiLU()
        self.outc = OutConv(64, out_ch)
    def forward(self, x, t, y):
        t_emb = self.time_mlp(timestep_embedding(t, self.time_mlp[0].in_features))
        y_emb = self.cond_mlp(y.float())
        emb   = t_emb + y_emb
        x1 = self.inc(x,   emb)
        x2 = self.down1(x1, emb)
        x3 = self.down2(x2, emb)
        x4 = self.down3(x3, emb)
        x5 = self.down4(x4, emb)
        x  = self.up1(x5, x4, emb)
        x  = self.up2(x,  x3, emb)
        x  = self.up3(x,  x2, emb)
        x  = self.up4(x,  x1, emb)
        x  = self.out_act(self.out_norm(x))
        out = self.outc(x)
        return out
```

UNetCond:

A UNet architecture is adopted, consisting of an encoder and a decoder, with skip connections to preserve feature information.

The model incorporates both time embedding (timestep converted into a sinusoidal vector and then processed by an MLP) and condition embedding (one-hot label processed by an MLP).

These embeddings are added together and fed into the Residual Block, where FiLM is applied to modulate the features.

The final output is a tensor with the same dimension as the input image, representing the predicted noise in DDPM.

DDPM:

The UNetCond is used as the denoising model, responsible for predicting noise at each timestep.

The image size is set to 64 × 64, with a total of T = 1000 diffusion steps.

Noise Schedule:

Linear and Cosine noise schedules are adopted. The Cosine schedule leverages the cosine function to preserve more information in the early timesteps while accelerating noise addition in the later stages.

```python
class DDPM(nn.Module):
    def __init__(self, model: nn.Module, image_size: int = 64, timesteps: int = 1000, beta_schedule: str = 'linear'):
        super().__init__()
        self.model = model
        self.image_size = image_size
        self.T = timesteps
        if beta_schedule == 'linear':
            betas = torch.linspace(1e-4, 0.02, self.T)
        elif beta_schedule == 'cosine':
            steps = torch.arange(self.T + 1, dtype=torch.float32)
            alphas_cumprod = torch.cos(((steps / self.T) + 0.008) / 1.008 * math.pi / 2) ** 2
            alphas_cumprod = alphas_cumprod / alphas_cumprod[0]
            betas = 1 - (alphas_cumprod[1:] / alphas_cumprod[:-1])
            betas = betas.clamp(1e-8, 0.999)
        else:
            raise ValueError('Unknown beta_schedule')
        self.register_buffer('betas', betas)
        alphas = 1.0 - betas
        alphas_cumprod = torch.cumprod(alphas, dim=0)
        self.register_buffer('alphas_cumprod', alphas_cumprod)
        self.register_buffer('sqrt_alphas_cumprod', torch.sqrt(alphas_cumprod))
        self.register_buffer('sqrt_one_minus_alphas_cumprod', torch.sqrt(1 - alphas_cumprod))
        self.register_buffer('alphas', alphas)
        self.register_buffer('alphas_prev', torch.cat([torch.tensor([1.0], dtype=torch.float32, device=alphas.device), alphas_cumprod[:-1]]))

    def q_sample(self, x0, t, noise=None):
        if noise is None:
            noise = torch.randn_like(x0)
        sqrt_ac = self.sqrt_alphas_cumprod[t].view(-1, 1, 1, 1)
        sqrt_om = self.sqrt_one_minus_alphas_cumprod[t].view(-1, 1, 1, 1)
        return sqrt_ac * x0 + sqrt_om * noise

    def p_losses(self, x0, t, y):
        noise = torch.randn_like(x0)
        xt = self.q_sample(x0, t, noise)
        noise_pred = self.model(xt, t, y)
        return F.mse_loss(noise_pred, noise)

    @torch.no_grad()
    def p_sample(self, x, t, y, guidance: float = 0.0):
        betas_t = self.betas[t].view(-1, 1, 1, 1)
        ac_t    = self.alphas_cumprod[t].view(-1, 1, 1, 1)
        ac_prev = self.alphas_prev[t].view(-1, 1, 1, 1)
        alpha_t = self.alphas[t].view(-1, 1, 1, 1)

        if guidance and guidance > 0:
            eps_c = self.model(x, t, y)
            eps_u = self.model(x, t, torch.zeros_like(y))
            eps   = eps_u + guidance * (eps_c - eps_u)
        else:
            eps = self.model(x, t, y)

        mean = (1 / torch.sqrt(alpha_t)) * (x - betas_t / self.sqrt_one_minus_alphas_cumprod[t].view(-1,1,1,1) * eps)
        sigma = torch.sqrt(betas_t * (1 - ac_prev) / (1 - ac_t))  # <-- correct std
        noise = torch.randn_like(x) if (t > 0).any() else torch.zeros_like(x)
        return mean + sigma * noise

    @torch.no_grad()
    def sample(self, batch_size, y, device, guidance: float = 0.0):
        x = torch.randn(batch_size, 3, self.image_size, self.image_size, device=device)
        for t in reversed(range(self.T)):
            t_tensor = torch.full((batch_size,), t, device=device, dtype=torch.long)
            x = self.p_sample(x, t_tensor, y, guidance=guidance)
        return x
```

train_loop:

Using parameters such as alphas_cumprod and sqrt_one_minus_alphas_cumprod, the signal-to-noise ratio at each timestep is computed.

Forward Diffusion:

The clean image is perturbed with random noise corresponding to the given timestep, resulting in a noisy image.

Training Loss:

The loss function is MSE Loss, which aims to minimize the difference between the true noise and the predicted noise.

Reverse Process:

During sampling, the model gradually reconstructs the image from random noise.

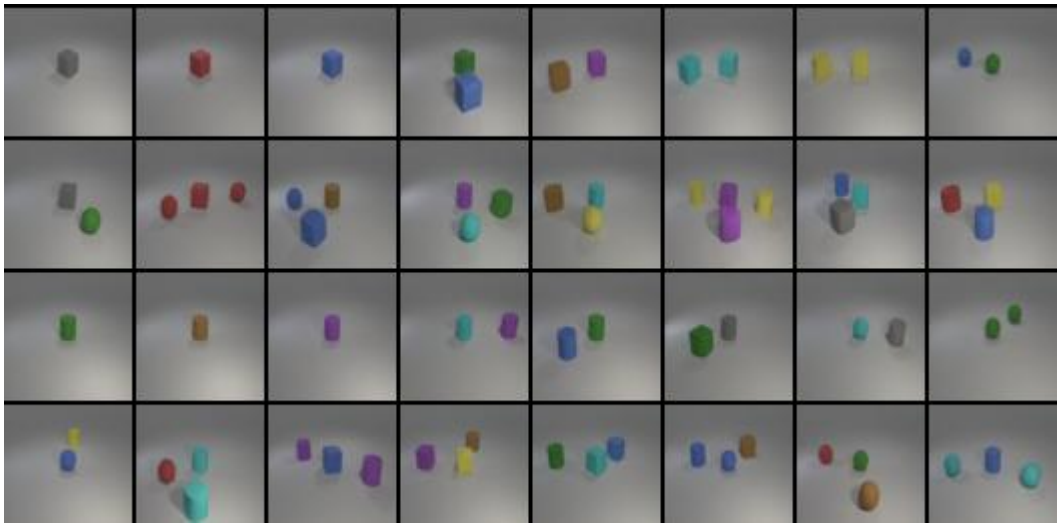
With CFG (Classifier-Free Guidance), eps_c denotes the conditional noise prediction, while eps_u represents the unconditional noise prediction.

Sampling:

Random Gaussian noise is initialized and iteratively denoised by reverse calls to p_sample, ultimately generating an image that satisfies the given condition.

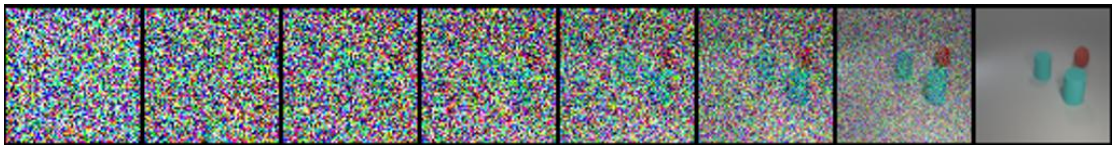
3. Results and discussion

Generated samples on test.json (accuracy = 80.56%)



Generated samples on new_test.json (accuracy = 86.90%)

Denoising process for labels



The model demonstrates better generation quality under single-object conditions, where both color and shape are mostly accurate. However, in multi-object scenarios, some objects exhibit color deviations or blurred shapes.

On test.json, the model achieves an accuracy of 80.56%, successfully generating most samples, though complex scenes still show inconsistencies in color or shape.

On new_test.json, the accuracy further improves to 86.90%, indicating that the model exhibits stronger generalization ability when facing novel condition combinations.

Experimental results:

Noise Schedule:

The Cosine schedule converges faster, produces smoother generated images, and achieves slightly higher accuracy compared to the Linear schedule.

CFG:

By adjusting the guidance scale, I found that values below 2 lead to higher diversity but insufficient condition alignment. At 4, condition alignment is strong but mode collapse tends to occur. The optimal range is between 2 and 3.

CBAM:

After integrating the CBAM attention module into the UNet block, the model produces more stable colors and shapes in multi-object scenarios.